PheW TuBe: Front End Engineering Viva Voce

Comprehensive Technical Questions & Model Answers Bank

Group G-15 (G15-PID-17) • Course: FEE-II (25CSE0203) • Chitkara University • Supervisor: Mr. Shubham Kumar

[1. HTML5 & Web Hyperlinks]

Q1: If a hyperlink is given, how do you make it download a file instead of navigating to or opening the URL in HTML?

Answer: By adding the download attribute to an HTML anchor (<a>) tag:

```
<a href="/assets/report.pdf" download="PheW_Report.pdf">Download File</a>
```

• **How it works:** The download attribute instructs the browser to download the target URL directly into the user's local disk rather than navigating to it.
• **Custom Filename:** Supplying a value (e.g., download="custom_name.pdf") specifies the default filename on the user's system.

[2. Core React Architecture & Reconciliation]

Q2: What is the difference between the Real DOM and the Virtual DOM? What is the core purpose of the Virtual DOM?

Answer:
• **Real DOM (Document Object Model):** The browser's native tree representation of UI elements. Direct DOM mutations are computationally expensive because each change triggers synchronous browser layout recalculation and full-tree repainting.
• **Virtual DOM:** An in-memory, lightweight JavaScript representation of the Real DOM managed by React.
• **Purpose & Mechanism:** When component state changes, React generates a new Virtual DOM snapshot, executes the **Diffing Algorithm (Reconciliation)** to compute the minimal set of differences (delta), and batches the exact updates to the Real DOM. This eliminates unnecessary layout re-renders and drastically boosts UI rendering performance.

[3. React State Management & Hooks]

Q3: What are React Hooks? Which specific hooks are used in the PheW TuBe project and why?

Answer: Introduced in React 16.8, Hooks are special built-in functions that enable functional components to utilize state and lifecycle features without writing ES6 class components.
• **useState:** Encapsulates local component state (e.g., active filter pills, search input values, modal visibility toggles).
• **useEffect:** Manages asynchronous side-effects (e.g., synchronizing state with browser LocalStorage, global key listeners).
• **useRef:** Persists a mutable reference to an actual DOM node without triggering component re-renders (e.g., auto-focusing the comment input field upon clicking Comment).
• **useContext:** Subscribes components to centralized global state (AppContext) without manually passing props across deep component trees (prop drilling).
• **useParams:** Reads dynamic URL parameters (e.g., extracting :id on route /watch/:id).
• **useNavigate:** Executes imperative, client-side route transitions programmatically.

[4. Package Management & Tooling]

Q4: How do you install third-party packages in a web project? What is the command, and which packages are installed?

Answer: Third-party packages are installed via **npm** (Node Package Manager), the default package registry for Node.js.
• **Command:** npm install <package-name> or shorthand npm i <package-name>.
• **Key packages in PheW TuBe:**
1. react-router-dom: Enables client-side declarative SPA routing (BrowserRouter, Routes, Route, useNavigate).
2. react-icons: Supplies lightweight SVG vector icons (YouTube icons, search, mic, thumbs up).
3. react-toastify: Renders non-blocking notifications (duplicate sign-up warning toasts, clear profile alerts).
4. react-infinite-scroll-component: Handles responsive infinite feed loading and dynamic DOM pagination.

[5. Dependency Management]

Q5: What is package.json and how does it differ from package-lock.json?

Answer:
• **package.json:** The root project manifest. It defines project metadata, runnable CLI scripts (dev, build, preview), and lists top-level project dependencies with semantic version ranges (e.g., ^18.2.0).
• **package-lock.json:** Automatically generated by npm upon installation. It locks down the exact recursive dependency graph, specific nested versions, and cryptographic integrity hashes (SHA-512). This guarantees 100% deterministic, identical installs across developer machines and production CI/CD builds.

[6. Developer Tooling & Code Snippets]

Q6: What is the shorthand snippet for React component boilerplate, and what CLI command scaffolds a new project?

Answer:
• **rafce:** *React Arrow Function Component with Export* (Generates a functional component using arrow syntax and exports it by default at the end).
• **rfce:** *React Functional Component with Export* (Generates a standard function declaration component with export).
• **Project Scaffolding Command:** npm create vite@latest <project-name> -- --template react (Scaffolds an optimized Vite + React project instantaneously).

[7. Styling Integration in React]

Q7: If you need to attach a CSS file to Navbar.jsx, how do you name it and link it into the component?

Answer:
• **File Naming:** Create a dedicated styling file named Navbar.css (or Navbar.module.css for scoped CSS Modules) in the same directory.
• **Import Syntax:** Place the import statement at the top of Navbar.jsx:
  import './Navbar.css';
• **CSS Modules Alternative:** If scoping is required to avoid global style collision:
  import styles from './Navbar.module.css'; and apply classes via className={styles.navbar}.

[8. Component Architecture & Reusability]

Q8: What is a React Component? How do you create and render a custom Navbar component inside another component?

Answer:
• **Component Definition:** A self-contained, reusable building block that returns JSX (HTML-like markup combined with JavaScript logic) and accepts dynamic inputs called **props**.
• **Creation:** In src/components/Navbar.jsx, declare and export the component:
  const Navbar = () => { return <nav className="navbar">...</nav>; }; export default Navbar;
• **Consumption:** In App.jsx, import and render it as a custom self-closing tag:
  import Navbar from './components/Navbar';
  function App() { return (<Navbar /><main>...</main>); }

[9. Object-Oriented Programming (OOP) in Front End Engineering]

Q9: How are Object-Oriented Programming (OOP) concepts applied in Front End Engineering and JavaScript?

Answer: Modern JavaScript implements OOP principles through ES6 Classes and underlying prototypal inheritance:
• **1. Classes & Objects:** class UserProfile { constructor(name) { this.name = name; } } — blueprint instantiation.
• **2. Inheritance:** Subclasses inherit behavior using the extends and super() keywords (e.g., class Creator extends UserProfile).
• **3. Encapsulation:** Bundling state with methods while keeping internal state private using ES private class fields (#privateId) or JS closures.
• **4. Polymorphism:** Subclasses can override parent methods to provide specialized rendering or data handling routines.

[10. JavaScript Array Mutation: slice vs. splice]

Q10: What is the functional difference between slice() and splice() in JavaScript? Why does this matter in React?

Answer:
• **slice(start, end):** Extracts a shallow copy of a portion of an array and returns a **new array** without mutating the original array (**Immutable**). Example: const topVideos = videos.slice(0, 8);.
• **splice(start, deleteCount, ...items):** Directly modifies the array by removing, replacing, or adding elements in place (**Destructive/Mutable**). Returns the array of deleted items.
• **Why it matters in React:** React state relies on **immutability** for reference checking during reconciliation. Directly mutating an array with splice() fails to trigger re-renders because the array reference remains identical. Therefore, immutable methods like slice(), filter(), and the spread operator ([...prev]) must always be used.

[11. DOM Interaction & Modern JavaScript (ES6+) Features]

Q11: How does PheW TuBe interact with the DOM, and what modern JavaScript (ES6+) features are implemented?

Answer:
• **React DOM Manipulation:** Rather than direct imperative manipulation (e.g., document.getElementById), React delegates DOM rendering to the React DOM reconciler. When direct imperative focus is necessary, the useRef() hook is cleanly attached to the input element without breaking the declarative cycle.
• **Modern ES6+ Features Employed:**
  • *Destructuring:* const { id } = useParams(); const { user, logout } = useApp();
  • *Spread Operator:* const updated = [...prevList, newItem]; for immutable state updates.
  • *Functional Array Methods:* .map() for rendering video grids, .filter() for voice search query matching.
  • *Async / Await & Promises:* Clean asynchronous control flow for data retrieval and audio recognition.
  • *Browser Web APIs:* **Web Speech API** (webkitSpeechRecognition) for speech recognition and **LocalStorage Web API** for persistence.

[12. Client-Side Persistence & Authentication Guards]

Q12: How does the application prevent duplicate user registrations, and how are user profiles managed in LocalStorage?

Answer:
• **Duplicate Registration Guard:** When a user submits the Sign Up form, AppContext queries the stored phew_registered_users array from localStorage. It evaluates whether the entered username or email already exists (case-insensitive). If found, it blocks registration, fires a warning toast (*'User already registered! Please sign in.'*), and automatically switches the active tab to Sign In.
• **LocalStorage Hydration:** Credentials and profile metadata are serialized via JSON.stringify() and retrieved via JSON.parse() upon application boot.
• **Testing Utilities:** A dedicated **'Clear All Saved Profiles'** button allows evaluators to reset stored users and test registration from a pristine state with one click.